

Collaborative Filtering Movie Recommender

User Based and Item Based CF with Sparse Rating Support

Student: Abaidullah Seikh Team: Individual

Email: abaidullahseikh06@gmail.com Date: 2026-09-29

Assignment: A03 Collaborative Filtering (Week 3)

Abstract

Implementing user based and item based collaborative filtering on MovieLens 100K, choosing one sparse rating strategy, and comparing quality, efficiency, cold start, and unreliable low overlap similarities. I implemented co rated cosine similarity with common rating support weighting, baseline deviation prediction, a Top 20 user neighborhood, cached item similarities, and side by side Top 5 recommendations in the starter interface. In a leakage checked 943 user holdout, User Based CF achieved MAE 0.8313 and 94.59% coverage, while Item Based CF achieved MAE 0.8071 and 99.79% coverage. The main lesson is that overlap support and cache behavior materially affect memory based CF, so accuracy, coverage, sparsity, and runtime must be interpreted together.

Index Terms: collaborative filtering, cosine similarity, MovieLens 100K, sparsity, cold start

1. Introduction

Problem statement. The Week 3 starter repository [1] provides a single page collaborative filtering recommender but leaves the rating matrix, cosine similarity, User Based CF, and Item Based CF as TODOs. The assignment requires both CF methods, one missing value strategy, and an analysis of quality, efficiency, cold start, and sparse overlap.

Motivation. MovieLens 100K contains 100,000 ratings over 943 users and 1,682 movies, so 93.70% of the user item matrix is missing. A high similarity can therefore be mathematically correct but supported by very little evidence, while a new user or item with zero ratings has no collaborative signal at all.

Overview. I followed a top down workflow: audit the starter and dataset, implement one explicit sparse rating strategy, verify lecture formulas and browser behavior, then run a leakage checked holdout, exhaustive overlap analysis, cold start fixtures, and cold/warm cache runtime benchmarks.

Concrete example. Users 1 and 88 share only *Air Bud (1997)*. Their raw cosine is 1.0 from that single overlap, but support weighting reduces it to 0.02, demonstrating why overlap count matters.

Contributions.

- 1 A verified User Based and Item Based Top 5 recommender using co rated cosine, common rating support weighting, and baseline deviation prediction.
- 2 A 943 user evaluation covering MAE/RMSE, prediction coverage, catalog exposure, exhaustive sparsity, synthetic cold start, and measured cache sensitive runtime.

2. Related Work and Alternatives

Neighborhood based collaborative filtering predicts preferences from users or items that behave similarly. GroupLens established user neighborhood CF [5], while Sarwar et al. formalized item based nearest neighbor recommendation and discussed quality, coverage, and scalability [6]. MovieLens is a standard recommendation benchmark family documented by GroupLens [3] and Harper and Konstan [4].

Alternatives considered. Mean imputation was not used because it fills missing cells with synthetic ratings and can inflate agreement. Matrix factorization can model latent structure and is a strong model based alternative [7], but it requires training, regularization, and hyperparameters beyond this memory based implementation. Weighted common rating support was selected because it directly addresses fragile low overlap similarity while remaining inspectable.

3. Method

3.1 Pipeline and Data Representation

Pipeline. u.item and u.data to validated parsing to raw ID user item matrix to observed only user/item means to weighted co rated cosine to User Based or Item Based baseline deviation prediction to candidate filtering to deterministic sorting to Top 5 output.

Component	Configuration
Dataset	MovieLens 100K: 1,682 movies, 100,000 ratings, 943 users
Missing value	0 in the matrix; excluded from means and similarity
Similarity	Cosine on common nonzero ratings; support $\min(n/50, 1)$
User neighborhood	Top 20 positive similarities
Top N	5 unseen movies, score descending; movie ID tie break
Prediction clipping	None; raw baseline deviation estimates retained

Tools & environment. Vanilla JavaScript; Bun 1.4.2; OpenCode 1.18.31; Chrome 153.0.8010.36; Linux x64 on Intel Core i7 to 6600U. The coding and experiment trail is preserved in the unmodified session.json.

3.2 Implementation Task: Similarity and Sparse Ratings

Raw similarity uses only the set C of co rated entries, so the zero used for a missing matrix cell never enters the dot product or norms. This follows the Week 3 memory based CF setup [2].

$$s_{raw}(a, b) = \frac{\sum_{k \in C} a_k b_k}{\sqrt{\sum_{k \in C} a_k^2} \sqrt{\sum_{k \in C} b_k^2}}, \quad n = |C|$$

Cosine measures rating pattern alignment only on entries observed for both entities.

$$s_w(a, b) = s_{raw}(a, b) \min\left(\frac{n}{50}, 1\right)$$

The n/50 factor is course derived; the cap at 1 is an implementation decision that preserves the starter similarity range.

Implementation area	Before	After
Rating matrix	TODO; null	944 x 1683 raw ID matrix; 100,000 observed cells
Missing values	0 placeholder	Ignored in cosine and means; never imputed
Similarity	TODO returns 0	Raw cosine + common count support weighting
Low overlap	Could look highly similar	n=1 gives at most 0.02; n>=50 gives full weight
Cold start	No handling	No evidence to no fabricated prediction
Reload	Arrays appended twice	Arrays and derived structures reset before parsing

Compact code evidence from the session log.

```
const supportWeight = Math.min(commonCount / 50, 1);
weightedSimilarity = rawCosine * supportWeight;
if (a[i] === 0 || b[i] === 0) continue;
return candidates.sort((a,b) => b.score-a.score || a.movieId-b.movieId).slice(0, topK);
```

3.3 Implementation Task: User Based and Item Based Prediction

User Based CF. The active user is compared with every other user, the 20 strongest positive neighbors are kept, and an unseen item is predicted from mean centered neighbor deviations. The active user mean is the baseline.

$$\hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in N_u(i)} s_w(u, v) (r_{vi} - \bar{r}_v)}{\sum_{v \in N_u(i)} |s_w(u, v)|}$$

A neighbor contributes only when it rated the candidate movie; zero support candidates are excluded.

Item Based CF. A candidate item is compared with movies already rated by the active user. The target item mean is the baseline, while the user's deviation from each rated item mean is transferred through item similarity.

$$\hat{r}_{ui} = \bar{r}_i + \frac{\sum_{j \in I_u} s_w(i, j) (r_{uj} - \bar{r}_j)}{\sum_{j \in I_u} |s_w(i, j)|}$$

The numerator keeps the signed contribution; the denominator uses absolute similarity as in prediction example.

- 1 **Distinct logic.** User Based compares users across movies; Item Based compares items across users.
- 2 **Repository interaction.** One active user produces two side by side Top 5 lists.
- 3 **Prediction range.** Scores are not silently clipped to 1 to 5 because clipping is not part of the specified formula.
- 4 **Caching.** Item similarities are cached; User Based similarities are recomputed per request.

4. Experiments

4.1 Browser and Baseline Comparison

Baseline. The starter executed data loading but could not recommend: buildRatingMatrix() left the matrix null, cosineSimilarity() returned 0, and both CF functions returned empty arrays. **After implementation.** All 34 executable test groups passed and a real Chrome run produced two five item lists for Users 1 and 33 with no normal console errors. Dataset SHA 256 values remained unchanged.

4.2 Quality Evaluation

Protocol. Quality was measured with one deterministic held out rating per user: the latest timestamp, breaking ties by the lowest movie ID. The target was removed from the rating records, matrix, user history, item rater index, observed only means, and relevant cache state before prediction, then all structures were restored. This is a per user latest holdout, not a globally chronological simulation.

$$MAE = \frac{1}{M} \sum_{t=1}^M |\hat{r}_t - r_t|, \quad RMSE = \sqrt{\frac{1}{M} \sum_{t=1}^M (\hat{r}_t - r_t)^2}$$

MAE/RMSE measure rating prediction error; coverage is the fraction of held out cases that receive a valid prediction.

Metric	User Based	Item Based
Attempted held outs	943	943
Valid predictions	892	941
Coverage	94.59%	99.79%
MAE	0.8313	0.8071
RMSE	1.0808	1.0360
Common subset MAE	0.8313	0.7942
Common subset RMSE	1.0808	1.0215

Comparison. The common predictable subset contains 892 cases. I report it because method level MAE/RMSE alone are not directly comparable when coverage differs. No statistical significance test was performed, so the error differences are descriptive rather than proof of universal superiority.

4.3 Recommendation Coverage and Ranking Behavior

Catalog exposure. Across all 943 full data Top 5 lists, User Based CF recommended 459 unique movies (27.29% of the catalog) and Item Based CF 29 (1.72%). Their lists were completely disjoint for 804 users and shared exactly one movie for 139 users. Wider catalog coverage is descriptive exposure, not automatically better recommendation quality.

4.4 Personal verification

Users 1 and 33 share four movies: *The Devil's Own* (245), *Contact* (258), *Event Horizon* (260), and *Starship Troopers* (271). Their co rated vectors are $A=[2,5,1,2]$ and $B=[3,4,4,4]$.

$$A \cdot B = 2(3) + 5(4) + 1(4) + 2(4) = 38, \quad \|A\|^2 = 34, \quad \|B\|^2 = 57$$

$$\cos(A, B) = \frac{38}{\sqrt{34}\sqrt{57}} \approx 0.86319, \quad w = \min\left(\frac{4}{50}, 1\right) = 0.08$$

$$S_{\text{weighted}} = 0.86319(0.08) \approx 0.06906$$

I independently reproduced the Users 1 and 33 similarity calculation from the four co rated movies. I obtained a dot product of 38, squared norms of 34 and 57, raw cosine similarity of approximately 0.86319, support weight of 0.08, and weighted similarity of approximately 0.06906. The result matched the executed test output.

4.5 Business Analysis: Sparsity and Cold Start

Exhaustive overlap. The study enumerated all distinct pairs: 444,153 user user pairs and 1,413,721 item item pairs. Small overlap is common, especially for items, and raw cosine can be deceptively high because positive rating vectors with one shared coordinate always have cosine 1.

Overlap evidence	User user pairs	Item item pairs	Raw median	Weighted median
0 common	15,043	430,515	0 guard	0
1 common	22,359	292,670	1.0000	0.0200
2 to 4 common	78,840	270,747	0.9701 / 0.9625	0.0579 / 0.0470
5 to 9 common	102,858	159,021	0.9515 / 0.9325	0.1275 / 0.1170
50+ common	41,709	39,459	0.9490 / 0.9461	0.9490 / 0.9461

Exact checks. Users 1 and 88 share one movie: raw cosine 1.0 becomes 0.02. Users 1 and 33 share four: 0.86319 becomes 0.06906. Users 1 and 5 share 80: support reaches 1, so the weighted value equals raw cosine 0.93261. Users 2 and 172 share none and return the zero overlap guard.

Cold start. Synthetic user 944 and item 1683 were inserted only in isolated test fixtures. Both had null observed means, zero common count, no useful similarity mass, and no CF prediction or Top 5 list: no ratings to no overlap to no similarity to no neighbors/evidence to no collaborative recommendation.

4.6 Business Analysis: Efficiency and Cache Behavior

$$\frac{943 \cdot 942}{2} = 444,153 \text{ user pairs}, \quad \frac{1682 \cdot 1681}{2} = 1,413,721 \text{ item pairs}$$

The item pair space is about 3.18 times larger here, but pair count alone does not determine request time speed.

Runtime protocol. Twenty deterministic users spanning history sizes were benchmarked with two warm ups and five timed repeats per method. performance.now() wrapped only the recommendation function. Cold cache runs cleared item similarities before each call; warm cache runs preserved them across repeats.

Scenario	User Based	Item Based	Interpretation
Cold median	169.72 ms	244.14 ms	Item comparisons are expensive when every similarity is recomputed
Cold mean	170.03 ms	302.79 ms	Item Based cold times are right skewed; max 895.47 ms
Warm median	172.45 ms	68.31 ms	Cached item similarities materially reduce repeated request cost
Warm mean	171.98 ms	92.92 ms	User Based has no analogous similarity cache

Interpretation. Neither approach is universally faster. If users greatly outnumber items, the structural comparison changes; within this implementation, warm item reuse is more important than the raw number of possible item pairs.

4.7 Missing Value Strategy Trade Off

Strategy	Simplicity	Bias / sparse behavior	Cost / fit
Mean imputation	High	Creates synthetic ratings; can pull similarity toward averages	Cheap formula but destroys sparsity; not selected
Weighted common ratings	High to moderate	Keeps observed only evidence and discounts weak overlap	One pass over overlap; transparent; implemented
Matrix factorization	Lower	Learns latent structure; depends on regularization and data support	Training/tuning required; outside this memory based implementation

4.8 Verification Evidence

- 1 **Implementation tests.** All 34 groups passed, including matrix integrity, cosine edge cases, support weights, candidate exclusion, deterministic ranking, lecture fixtures, cold start, cache invalidation, and dataset hashes.
- 2 **Cross check.** The Week 3 examples reproduced approximately 3.05 for User Based and 2.58 for Item Based prediction.
- 3 **Evaluation harness.** It evaluated 943 held outs, all 444,153 user pairs, all 1,413,721 item pairs, all 943 Top 5 outputs, and 400 timed calls; validation.json recorded passed=true.
- 4 **Browser check.** Chrome verification covered users 1 and 33, both two column Top 5 outputs, and no normal console errors.

5. Discussion

5.1 Failure Mode Verification

Leakage check. For each held out case the target was removed from every structure used by prediction and restored afterward; 943 checked and 943 restored cases were recorded. **Range check.** Production predictions were intentionally unclipped: 3.14% of valid User Based held out estimates and 0.32% of Item Based estimates fell outside 1 to 5. **Coverage check.** User Based missed 51 targets because no Top 20 neighbor rated the target; Item Based missed two single rater items after that sole rating was held out.

5.2 Failure Case, Root Cause, Fix, and Confirmation

Failure case. The baseline loadData() appended records on repeated loading: a second load produced 3,364 movie records and 200,000 ratings instead of 1,682 and 100,000.

Root cause. Global movies and ratings arrays were never cleared, so each load duplicated every record and would corrupt later means, similarity counts, and recommendations.

Fix and verification. I reset the arrays, matrix, dimensions, and derived indexes before parsing. A regression test then called loadData() again and confirmed 1,682 movies, 100,000 ratings, and identical real Item Based recommendations for User 1.

5.3 What Worked and What Surprised Me

- 1 **What worked.** Separating raw cosine from common rating support made one overlap pairs drop from 1.0 to 0.02 instead of appearing perfectly reliable.
- 2 **What worked.** A shared baseline deviation helper kept both prediction equations consistent while preserving distinct user user and item item similarity logic.
- 3 **Surprise.** The strongest quality difference was coverage (94.59% vs. 99.79%), not the small MAE gap.
- 4 **Surprise.** Item Based was slower cold but much faster warm, while User Based changed little because it has no similarity cache.
- 5 **Surprise.** Top 5 catalog exposure differed sharply (459 vs. 29 unique movies), showing that similar rating error does not imply similar recommendation behavior.

5.4 Interpretation, Limitation, and Next Improvement

Interpretation. Common rating weighting is a practical compromise because it preserves observed only evidence and directly weakens fragile low overlap similarity, but it does not solve true cold start. Mean imputation inserts assumptions into 93.70% missing cells; matrix factorization can capture latent structure but introduces training and tuning rather than transparent memory based reasoning.

Limitation. The latest rating is held out per user, but other users' later interactions can remain in training. There is one target per user, no repeated split or significance test, and MAE/RMSE evaluate rating prediction rather than ranking relevance.

Next improvement. With more time I would use a globally chronological evaluation with Recall@K or NDCG, then test an explicitly justified calibration/clipping policy for scores outside 1 to 5. I would also investigate why Item Based Top 5 output concentrates on 29 movies despite high held out coverage.

5.5 Conclusion

The final system satisfies the Week 3 implementation task: it builds the user item matrix, computes support weighted co rated cosine, implements distinct User Based and Item Based baseline deviation predictions, and returns two deterministic Top 5 lists. The evaluation shows that Item Based and User Based CF differ not only in error and coverage but also in catalog concentration and cache sensitive runtime, while sparse overlap and zero history cold start remain structural limitations.

6. AI Usage Disclosure

AI tools used. OpenCode 1.18.31 was used for baseline inspection, code generation/modification, debugging, executable tests, browser verification scripts, and the Step 3 evaluation harness.

Verified through executed session evidence. Matrix construction, cosine/support edge cases, both prediction formulas, 34 implementation test groups, real browser behavior, 943 held outs, exhaustive pair counts, cold start, runtime summaries, and unchanged dataset hashes were executed and preserved in the session.

Personal verification status. I independently reproduced the Users 1 and 33 hand calculation in Section 4.4 and confirmed that it matched the executed test output.

References

- [1] S. Jin, "RecSys LLMs: Week 3 starter repository," GitHub. [Online]. Available: <https://github.com/dryjins/RecSys-LLMs/tree/main/week3>. [Accessed: 2026-09-29].
- [2] S. Jin, "LLM4RecSys 03. Collaborative Filtering," Week 3 lecture slides, HSE University, 2026.
- [3] GroupLens Research, "MovieLens 100K Dataset," University of Minnesota. [Online]. Available: <https://grouplens.org/datasets/movielens/100k/>. [Accessed: 2026-09-29].
- [4] F. M. Harper and J. A. Konstan, "The MovieLens Datasets: History and Context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, Art. 19, 2015, doi: 10.1145/2827872. [Online]. Available: <https://doi.org/10.1145/2827872>. [Accessed: 2026-09-29].
- [5] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "GroupLens: An Open Architecture for Collaborative Filtering of Netnews," in *Proc. ACM Conf. Computer Supported Cooperative Work (CSCW)*, 1994, pp. 175 to 186, doi: 10.1145/192844.192905. [Online]. Available: <https://doi.org/10.1145/192844.192905>. [Accessed: 2026-09-29].
- [6] B. M. Sarwar, G. Karypis, J. A. Konstan, and J. Riedl, "Item based Collaborative Filtering Recommendation Algorithms," in *Proc. 10th Int. Conf. World Wide Web (WWW)*, 2001, pp. 285 to 295, doi: 10.1145/371920.372071. [Online]. Available: <https://doi.org/10.1145/371920.372071>. [Accessed: 2026-09-29].
- [7] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30 to 37, 2009, doi: 10.1109/MC.2009.263. [Online]. Available: <https://doi.org/10.1109/MC.2009.263>. [Accessed: 2026-09-29].